

**SC4020 DATA ANALYTICS & MINING**

**Project 1**

**Review Report**

**Submitted to: Lin Guosheng (gslin@ntu.edu.sg)**

**Submitted by: Group 47**

|                         |           |
|-------------------------|-----------|
| Shubhangam Prasad       | U2129974E |
| Bhati Nancy             | U2222368G |
| Neha Ramesh             | U2023864F |
| Trisha Bankata Mishra   | U2222358J |
| Prakritipong Phuvajakrt | U2222277F |

## **Abstract**

This project delves into the topic of clustering methods and analyses the performance of various clustering methods: K-means, DBSCAN, Bayes Net, on 3 datasets using metrics such as silhouette score and the Davies-Bouldin score. The strengths and weaknesses of each method are evaluated to gain insight into how different clustering methods respond to varying data structures.

## **Introduction**

Clustering is a key technique in unsupervised machine learning, commonly used to uncover patterns and groupings within datasets. With various algorithms available, clustering has applications across numerous fields such as image recognition and anomaly detection. This project centers on analyzing and comparing three widely used clustering methods: K-means, DBSCAN, and Bayesian Networks, specifically in the context of customer segmentation. Each method employs a distinct approach to defining clusters, making them suitable for different types of datasets.

The aim of this report is to apply these three methods to three datasets, analyze the results, and assess their performance using standard clustering evaluation metrics, including the silhouette score. By examining the strengths and limitations of each algorithm, this report will help identify the most suitable clustering technique based on the unique characteristics of each dataset.

## **Datasets**

The first dataset taken from Kaggle is called the 2004 NBA Shot report, which will be referred to as Dataset 1. This dataset has 26 columns and 189803 rows. The rows were cleaned, looking at those that can be most relevant to determine the probability of a shot successfully being taken. There are no null values or duplicates.

The second dataset is taken from Kaggle named World Happiness Report. This dataset will be referred to as Dataset 2 thereafter. We have used the 2019 Happiness ranks and scores by countries. It comprises of 9 columns and 156 rows. This dataset is selected as it is relatively small and easy to handle with no null values or duplicates.

|   | Overall rank | Country or region | Score | GDP per capita | Social support | Healthy life expectancy | Freedom to make life choices | Generosity | Perceptions of corruption |
|---|--------------|-------------------|-------|----------------|----------------|-------------------------|------------------------------|------------|---------------------------|
| 0 | 1            | Finland           | 7.769 | 1.340          | 1.587          | 0.986                   | 0.596                        | 0.153      | 0.393                     |
| 1 | 2            | Denmark           | 7.600 | 1.383          | 1.573          | 0.996                   | 0.592                        | 0.252      | 0.410                     |
| 2 | 3            | Norway            | 7.554 | 1.488          | 1.582          | 1.028                   | 0.603                        | 0.271      | 0.341                     |
| 3 | 4            | Iceland           | 7.494 | 1.380          | 1.624          | 1.026                   | 0.591                        | 0.354      | 0.118                     |
| 4 | 5            | Netherlands       | 7.488 | 1.396          | 1.522          | 0.999                   | 0.557                        | 0.322      | 0.298                     |

Figure 1. World Happiness Report

The 9 features of the dataset are Overall rank, Country or region, Score, GDP per capita, social support, Healthy life expectancy, Freedom to make life choices, Generosity and Perceptions of corruption.

The third dataset we used was taken from Kaggle called Mall Customer Segmentation Data. This dataset contains the Gender, CustomerID, Spending Power, Annual Income and Age of the users of supermarket and mall membership cards. The aim of using this dataset is to be able to categorize different types of users based on their attributes to analyze their purchasing habits and design optimal marketing strategies. It contains 200 rows and 5 columns while also containing no null values.

### Evaluative Criteria

In this report, the performance of the clustering algorithms will be evaluated using two key metrics: the Silhouette Score and the Davies-Bouldin Index (DBI). These metrics will provide an assessment of how well each algorithm performs across the three datasets. After analyzing the results, we will identify the best-performing methods for each dataset and provide a detailed explanation of the factors contributing to their success. This analysis will offer valuable insights into why certain algorithms are more effective than others based on the characteristics of the datasets.

### Methods

We decided to compare three clustering methods- K-means, DBSCAN and Bayesian Networks.

#### K-Means

K-Means clustering is a partitioning method that divides data into K clusters based on their distance from cluster centroids. The algorithm initializes by randomly selecting K centroids, which are the central points of each cluster. Data points are assigned to the nearest centroid, and the centroids are updated iteratively by calculating the mean of the points in each cluster. This process repeats until the centroids no longer move

significantly, indicating convergence. K-Means is efficient for clustering large datasets, but it assumes clusters are spherical and can be sensitive to the initial placement of centroids.

#### Strengths:

- It is simple to implement and computationally efficient, especially on large datasets.
- It works well when clusters are spherical and well-separated.
- It assigns every point to a cluster, ensuring that each data point has a label.
- The centroids represent the centre of the clusters, making it easy to understand and interpret the clustering results.

#### Weaknesses:

- The result depends heavily on the initial placement of the centroids, and poor initialization can lead to suboptimal results.
- It assumes that clusters are spherical and equally sized, which limits its applicability to datasets with complex or elongated clusters.
- Outliers can significantly affect the centroids, leading to inaccurate clusters.

### **DBSCAN**

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a clustering algorithm that groups data points based on density. It identifies clusters as areas of high point density, with clusters being separated by regions of lower density. The algorithm works by expanding clusters from points that have a minimum number of neighboring points within a given distance (epsilon). Points that do not meet this criterion are considered noise or outliers. Unlike K-Means, DBSCAN does not require the number of clusters to be specified in advance and can handle clusters of arbitrary shape and size, making it robust to outliers.

#### Strengths

- It can detect clusters of arbitrary shapes, making it suitable for complex datasets where clusters are not necessarily spherical.
- It can identify noise (outliers) and assign them to a separate group, making it robust to noise in the data.
- Unlike K-Means, DBSCAN does not require specifying the number of clusters in advance. Instead, it relies on density parameters.

#### Weaknesses:

- Its performance depends heavily on two parameters: `eps` (the neighbourhood radius) and `min_samples` (minimum number of points in a neighbourhood to form a cluster). Poor parameter choices can lead to poor clustering results.
- It struggles when clusters have varying densities, as a fixed `eps` may not work well for all clusters.
- Its time complexity is higher than K-Means, and it may struggle with large, high-dimensional datasets as it is not scalable
- The concept of density in DBSCAN is harder to define in high-dimensional spaces, where distances between points become less meaningful.

## **Bayesian Network**

Bayesian Network Clustering is a probabilistic method that groups data points based on their conditional dependencies. It uses a graphical model, where nodes represent variables and edges indicate probabilistic relationships between them. The algorithm clusters data by constructing a network for each cluster, capturing how features are related within that group. Data points are assigned to clusters based on their likelihood under the network's structure. This approach is particularly useful for capturing complex, interdependent relationships in the data while allowing for a probabilistic interpretation of clusters.

### **Strengths:**

- It explicitly models probabilistic relationships between variables, making them ideal for clustering in situations where understanding the dependencies is critical.
- It can handle missing data effectively by making probabilistic inferences, which is useful in real-world scenarios with incomplete information.

### **Weaknesses:**

- Learning a Bayesian network and performing inference in it can be computationally expensive, especially in large or high-dimensional datasets.
- Accurate probabilistic modeling in Bayesian networks often requires large datasets, especially when dealing with complex dependencies between variables.
- The success of Bayesian networks relies on the assumption of conditional independence between variables. If this assumption is not true, clustering may not be accurate.

## Experiments

To conduct our experiment to determine the strengths and weaknesses of each algorithm we will be implementing them on the datasets stated above. This exposes the algorithm to different types of data and their respective distributions to allow us to observe the differences in their performances in varying scenarios. At the end of the experiment, we will compare their performances using standard metrics like Silhouette Score and Davies-Bouldin Index. The experiments are as follows.

### DATASET 1

The first dataset taken from Kaggle is called the NBA Shot report. The dataset consists of detailed shot data, including attributes such as position groups, shot types, distances, and zones. The aim of the analysis is to group similar shots based on these attributes using K-means, DBSCAN, and Bayesian clustering techniques.

#### Approach 1: K-means Clustering

##### Data Preprocessing:

Feature Selection: We selected columns related to the shot types and distances for clustering, including fields like SHOT\_TYPE, SHOT\_DISTANCE, ZONE\_RANGE, and POSITION\_GROUP.

Standardization: The selected columns were standardized using StandardScaler to ensure uniform contribution of all features in the distance calculations.

##### Clustering Process:

K-means was applied with  $n\_clusters=5$ , and the dataset was partitioned into five distinct clusters.

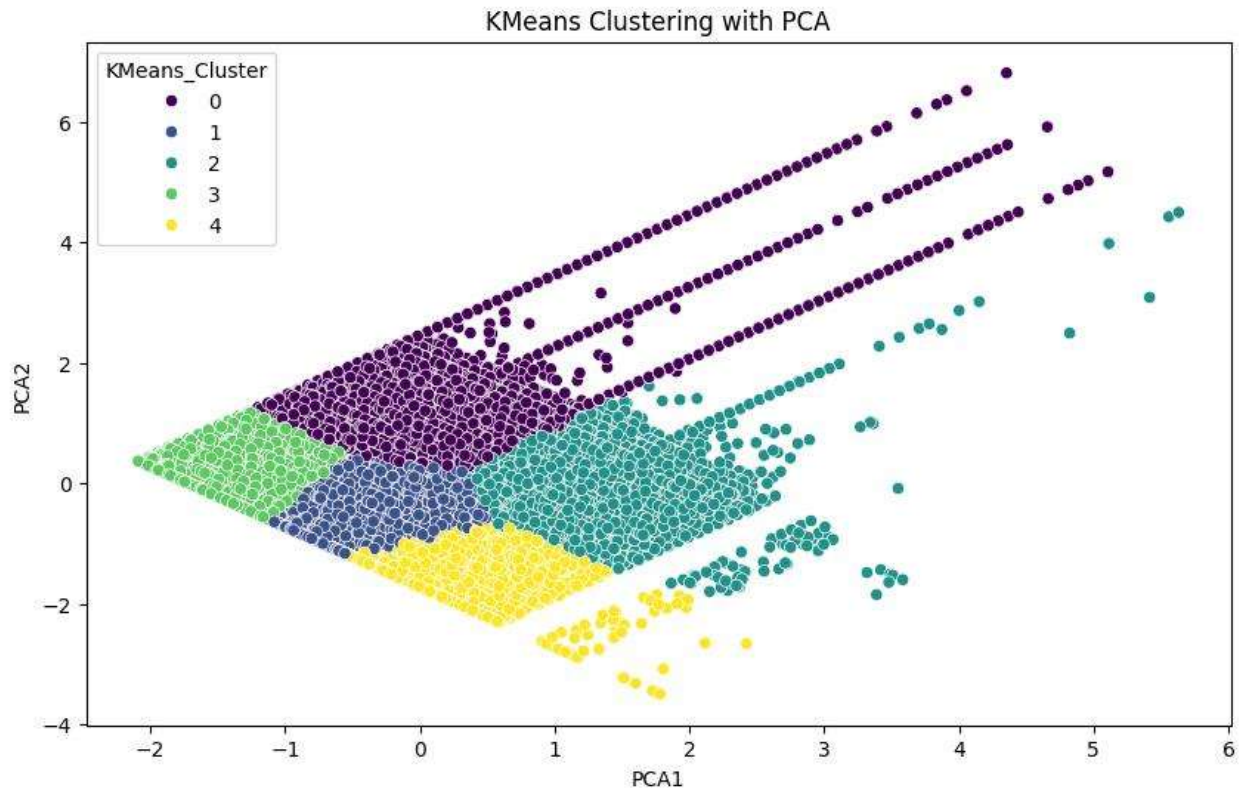
##### Results:

Each cluster was characterized by its unique shot attributes. For instance, one cluster might represent close-range shots from the paint, while another could focus on long-distance attempts.

##### Performance Evaluation:

Silhouette Score: The silhouette score of 0.42 suggested moderate clustering quality, indicating that there is clear, though not perfectly separated, clusters.

Davies-Bouldin Index (DBI): A DBI score of 0.84 further corroborated the moderate quality of the clustering.



K Means Clustering visualization using PCA

## Approach 2: DBSCAN Clustering

### Data Preprocessing:

- The same feature set used for K-means was applied here, focusing on shot type, shot distance, and basic zones.
- StandardScaler was used for data normalization.

### Clustering Process:

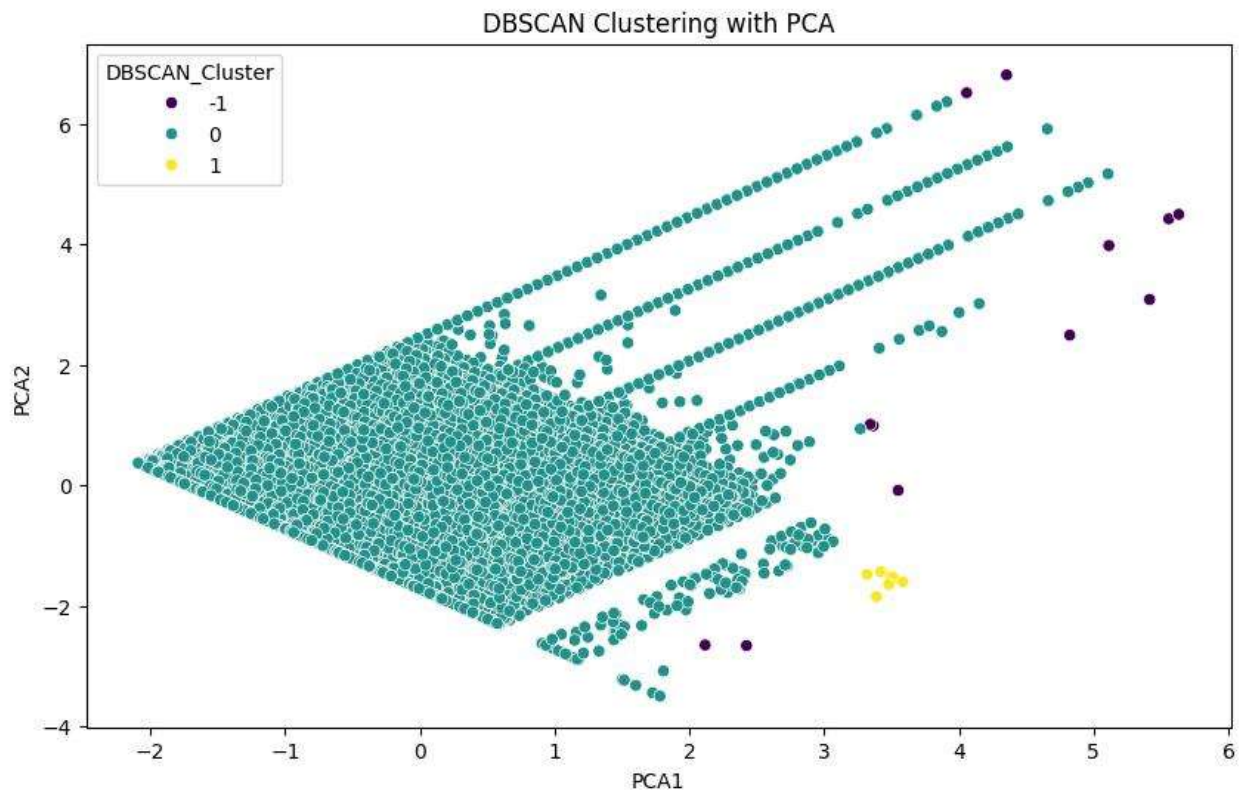
1. DBSCAN was executed with  $\text{eps}=0.5$  and  $\text{min\_samples}=5$ .
2. The algorithm identified several clusters and marked outliers (e.g., shots that didn't fit well into any cluster) as noise points.

### Results:

- DBSCAN revealed clusters of shots with similar distances and zone classifications. It also flagged some noise, particularly in cases where shot attempts didn't conform to typical patterns.

### Performance Evaluation:

- **Silhouette Score:** A silhouette score of 0.51, excluding noise points, indicated well-separated clusters.
- **Davies-Bouldin Index (DBI):** The DBI score of 0.78 confirmed that the clusters were compact and distinct, showcasing excellent clustering performance.



### Approach 3: Bayesian Clustering

#### Data Preprocessing:

- Bayesian methods were applied using the same standardized data as the previous approaches.

#### Clustering Process:



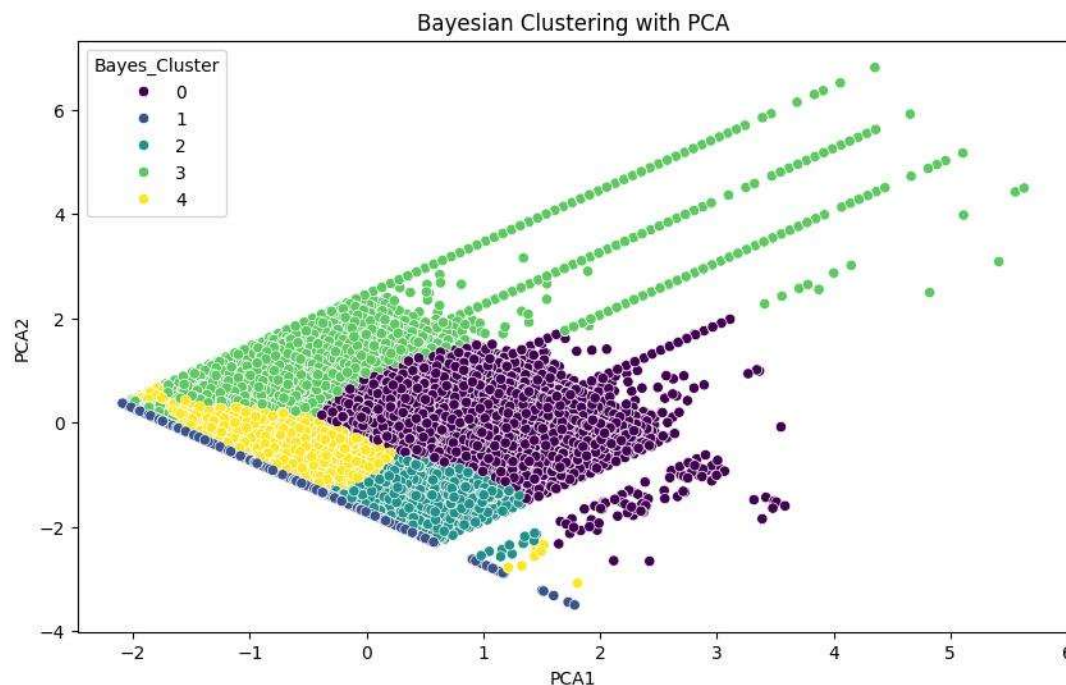
1. Bayesian clustering was performed, and after experimenting with different numbers of clusters, it was determined that two clusters offered the best balance between separation and cohesion.

## Results:

- The Bayesian method grouped shots based on probabilistic relationships. For example, one cluster might represent shots with higher distances and lower accuracy, while another might focus on close-range, high-percentage shots.
- A 3D plot was used to visualize the clusters.

## Performance Evaluation:

- **Silhouette Score:** The silhouette score of 0.31 suggested moderately defined clusters.
- **Davies-Bouldin Index (DBI):** The DBI score of 1.3 indicated weak clustering performance, with some overlap between the clusters.



## Bonus:

To try and improve the score, we attempted to adjust various parameters such as reducing

sample size using PCA, adjusting the eps number and removing noise for DBSCAN. We saw notable improvements in the scores as a result. PCA caused Bayes scores to worsen.

| Method              | Initial Silhouette Score | Final Silhouette Score | Initial DBI Score | Final DBI Score |
|---------------------|--------------------------|------------------------|-------------------|-----------------|
| K Means             | 0.238                    | 0.402                  | 1.51              | 0.835           |
| DBSCAN              | 0.201                    | 0.505                  | 1.81              | 0.785           |
| Bayesian Clustering | 0.306                    | 0.175                  | 1.30              | 1.56            |

### Insights:

- **K-means:** Provided a useful overview of shot groupings, particularly for identifying general patterns. However, some clusters overlapped, likely due to the complex nature of shot distributions.
- **DBSCAN:** Excelled in finding distinct clusters and identifying outliers, making it well-suited for separating uncommon shot types or extreme distances.
- **Bayesian:** While it captured nuanced relationships, it struggled with clearly separated clusters, reflecting the subtle dependencies between shot attributes.

### Recommendations:

- **For future analyses,** DBSCAN seems to offer the most robust clustering, especially for distinguishing uncommon shot types and outliers.
- **Alternative algorithms** could be explored if K-means continues to yield moderate scores, as other methods may better handle the complexities of shot data. Unsupervised learning may not be the ideal approach here, doing a weighted Bayes Net might be better in helping us understand what factors affect the probability of a shot occurring.

## DATASET 2

The World Happiness Report dataset from Kaggle contains data on global happiness scores for various countries, based on factors like GDP, social support, life expectancy, freedom, and corruption perceptions. It provides insights into the relationships between these factors and the overall happiness levels of different nations from multiple years.

On this dataset, we decided to perform K-means, DBSCAN and Bayes Net.

## Approach 1: K-means

K-means Clustering of Countries based on Happiness Score and GDP (2019)

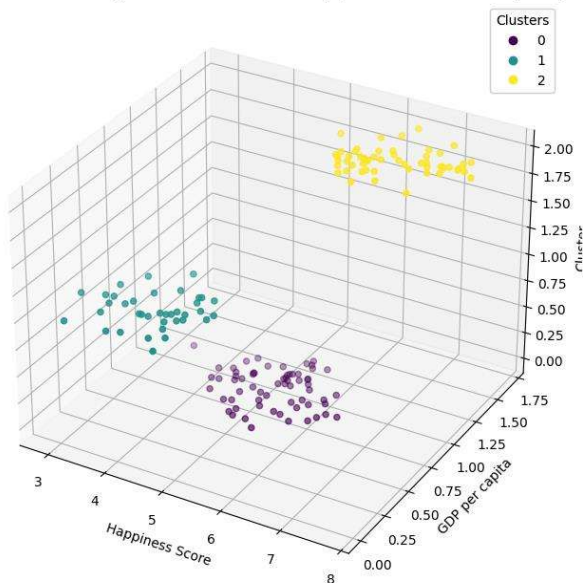


Fig: K-Means clustering for our dataset

### Data Preprocessing:

1. Data Selection: We selected relevant columns from the dataset to assess the relation between the parameters- Happiness Score, Country, and GDP per capita.
2. Data Standardization: The 3 chosen columns are scaled using StandardScaler. This normalization standardizes the range of continuous initial variables so that each feature contributes approximately proportionately to the final distance computations in the algorithm.

### Data Visualization:

To effectively visualize the data, we needed to calculate the number of clusters first. This was done by plotting the elbow graph to calculate the inertia for the number of clusters. From the graph, 3 clusters were chosen to be the optimal number of clusters. We then carried out K-means clustering using the three factors, to understand the correlation between the factors. We plotted a 3-dimensional graph for the same. Each point's color represents its cluster label.

## **Plot Analysis**

### **1. Visualization Details:**

Data points are colored based on their cluster membership with different colors indicating different clusters.

### **2. Interpreting Clusters:**

The clusters reveal groupings based on the happiness score and GDP per capita of different countries. For instance, a cluster of yellow points might indicate a segment of countries with higher GDP per capita and a higher happiness score, while green points might indicate countries with lower GDP per capita and a lower happiness score. K-means clustering was able to effectively cluster the data points.

3. Implications of the Results: The detected clusters can help each country's governments tailor strategies based on the GDP per capita and the happiness score. They can strategize to improve the GDP.

## **Clustering Analysis**

### **1. K means Instantiation:**

The script initializes a K means algorithm with the number of clusters set to 3. This was chosen by utilizing the elbow method, to calculate the inertia for different numbers of clusters.

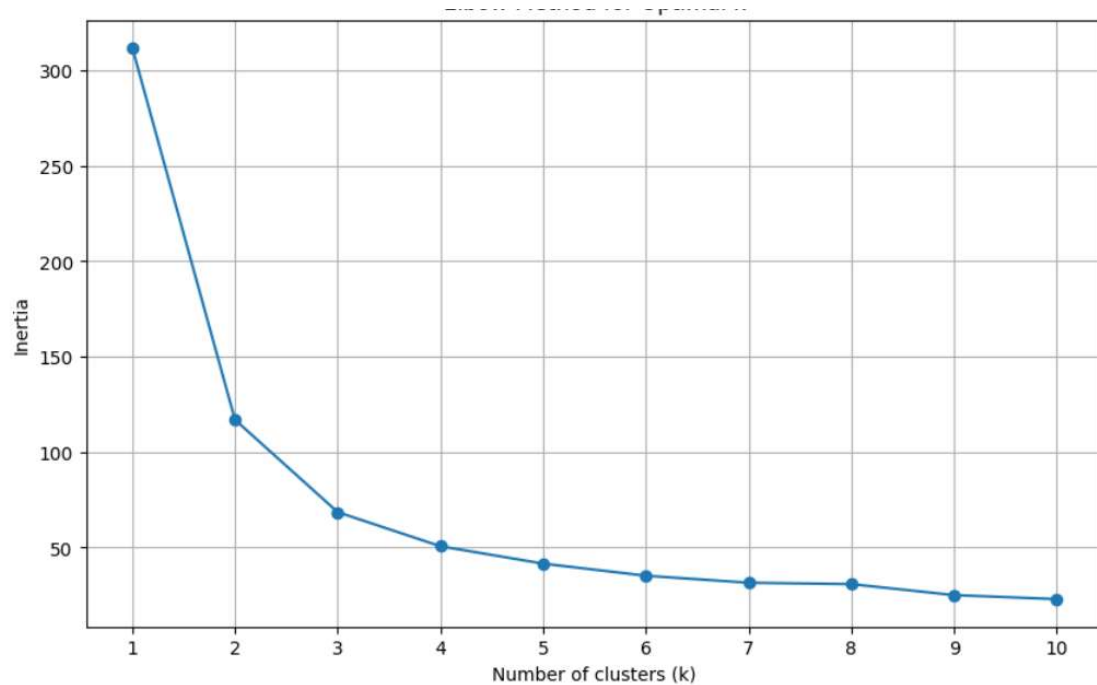


Fig: Elbow Plot

## 2. Cluster Assignment:

Once the model is trained, each data point (i.e., each country) is assigned to one of the three clusters based on the distance to the cluster centroids. The centroid represents the mean position of all the points in that cluster.

## 3. Plotting:

To visualize the clustering, a 3D scatter plot is generated, by utilizing the K-means clustering using the three factors, to understand the correlation between the factors. We plotted a 3-dimensional graph for the same. Each point's color represents its cluster label.

## Evaluation of Clustering Performance:

**Silhouette Score:** To evaluate the clustering quality, the silhouette score was calculated, which came out to be 0.4472718292353524, which is a moderate score.

**Davies-Bouldin Index (DBI):** The DBI score was calculated to be 0.7461020768092844, which is a moderate score.

## Approach 2: DBSCAN

DBSCAN Clustering of Countries based on Happiness Score and GDP (2019)

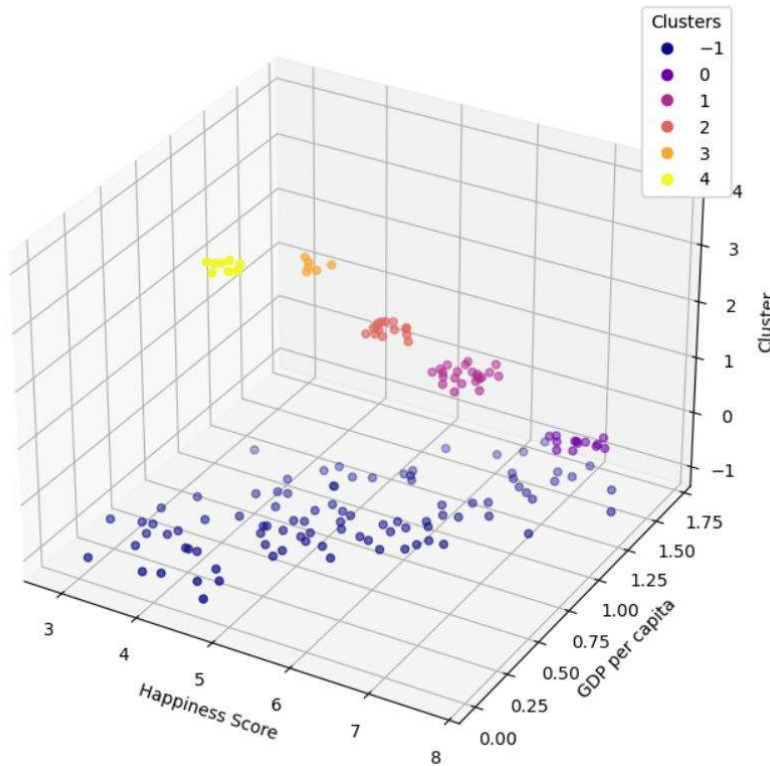


Figure: DBSCAN Clustering of Countries based on Happiness Score and GDP (2019)

### DBSCAN Clustering

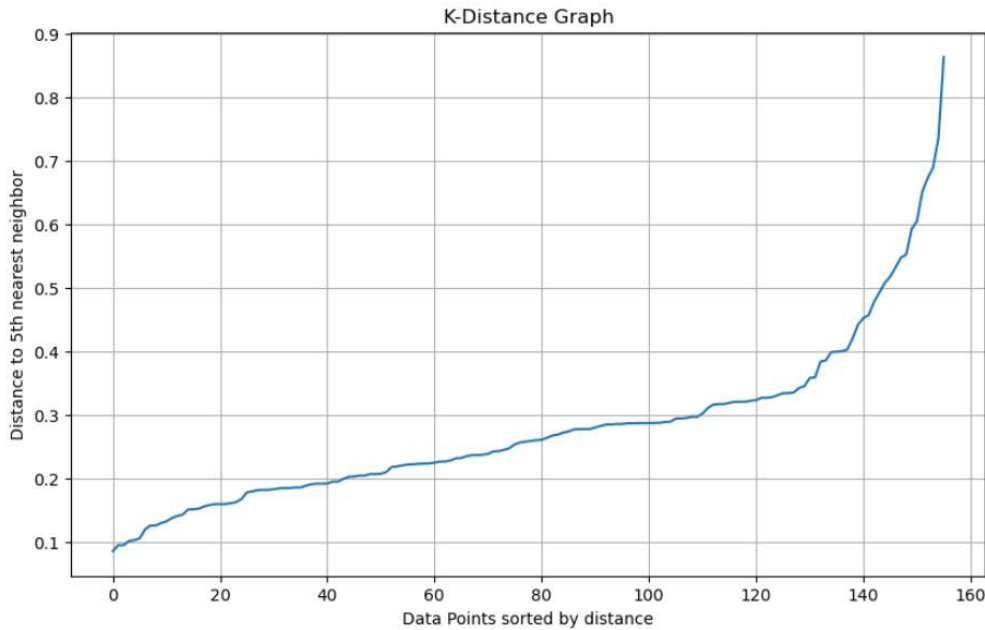
#### 1. Data Preprocessing:

**Feature Selection:** For the clustering analysis, we focused on the columns: 'Score' (Happiness Score) and 'GDP per capita'.

**Standardization:** Before applying DBSCAN, the data was standardized using StandardScaler to ensure all features contribute equally to the distance calculations in DBSCAN.

#### 2. Determining eps and min\_samples for DBSCAN:

**K-Distance Graph:** To find an optimal value for eps, a k-distance graph was plotted using the distances to the 5th nearest neighbor (min\_samples=5). The knee point of the graph was identified around 0.35, but as a common practice, a slightly lower value of eps=0.2 was chosen.



### 3. Data Visualization:

**DBSCAN Parameters:** The DBSCAN algorithm was applied with  $\text{eps}=0.2$  and  $\text{min\_samples}=5$ .

**Cluster Labels:** DBSCAN successfully grouped the countries into distinct clusters and marked several outliers (noise points) with the label -1.

**Plotting the results:** A 3D scatter plot was created to visualize the clusters. The axes represent Happiness Score, GDP per capita, and the assigned Cluster label.

### 4. Results and Visualization:

**Cluster Assignment:** The resulting cluster labels were added to the dataset, revealing which countries belong to the same cluster based on their happiness score and GDP per capita. Some countries were marked as noise due to sparse data points.

**3D Visualization:** Each color represents a distinct cluster, while noise points are labeled as -1 and are visualized separately.

### 5. Evaluation of Clustering Performance:

**Silhouette Score:** To evaluate the clustering quality, the silhouette score was calculated after excluding noise points. The silhouette score of 0.6842 suggests that the clusters are well-separated and distinct, indicating good clustering performance.

**Davies-Bouldin Index (DBI):** The DBI score of 0.3804 indicates that the clusters are highly distinct and compact, with a score below 0.5 being considered very good.

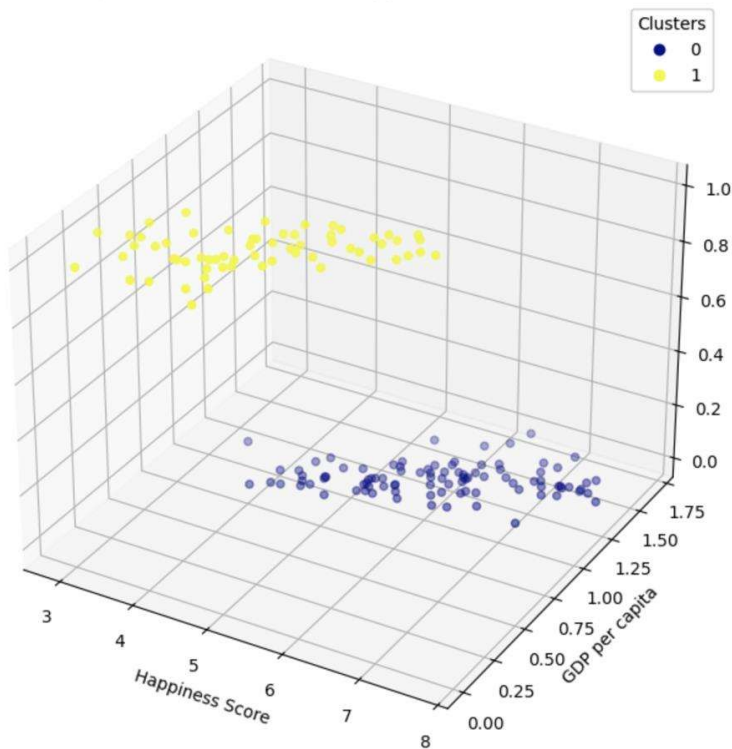
## **6. Insights from the Results:**

**Cluster Analysis:** The clusters formed by DBSCAN revealed distinct groupings of countries based on happiness score and GDP per capita. Countries with high GDP per capita and high happiness scores tend to form separate clusters, while outliers (e.g., countries with lower happiness and GDP) were labeled as noise.

## **Approach 3: Bayes Net**



Clustering of Countries based on Happiness Score and GDP (2019)



## 1. Data Preprocessing:

**Feature Selection:** For the clustering analysis, we focused on the columns: 'Country', 'Score' (Happiness Score) and 'GDP per capita'.

**Standardization:** Before applying Bayes Net, the data was standardized using MinMaxScaler to ensure all features contribute equally to the distance calculations in the clustering.

## 2. Data Visualization:

**Cluster Labels:** Bayes Net successfully grouped the countries into two distinct clusters. The number of clusters were experimented with to maximize silhouette score and DBI. The best clustering results were shown when two clusters and seven clusters were formed. We have picked two as the clusters formed were better than with seven clusters.

**Plotting the results:** A 3D scatter plot was created to visualize the clusters. The axes represent Happiness Score, GDP per capita, and the assigned Cluster label.

## 3. Results and Visualization:

**Cluster Assignment:** The resulting cluster labels were added to the dataset, revealing which countries belong to the same cluster based on their happiness score and GDP per capita.

#### 4. Evaluation of Clustering Performance:

**Silhouette Score:** The silhouette score of 0.41 suggests that the clusters are moderately separated and distinct, indicating moderate clustering performance.

**Davies-Bouldin Index (DBI):** The DBI score of 0.84 indicates that the clusters are fairly distinct. As a score below 0.5 is considered very good, this clustering can be assumed to be moderate.

#### 5. Insights from the Results:

**Cluster Analysis:** The clusters formed by Bayes Net revealed distinct groupings of countries based on happiness score and GDP per capita. These clusters can be assumed to have a strong probabilistic relationship within itself. For example, the points in a cluster could be assumed to be countries that have a higher GDP and/or Happiness Score.

#### Silhouette Score Comparison:

The silhouette score is a measure of cluster cohesion and separation. It ranges from -1 (incorrect clustering) to +1 (highly dense clustering), with scores around zero indicating overlapping clusters.

#### 1. K-means

We obtained a moderate silhouette score of 0.45.

Reasons for this could include:

**Overlapping Clusters:** Happiness Score and GDP per capita are correlated but not perfectly. There may be several countries with similar happiness levels but differing economic performance, or vice versa. For example, some middle-income countries might have a high happiness score, making it harder for K-means to cleanly separate them into distinct clusters.

**K-Means limitations:** In datasets like the World Happiness Report, clusters might be irregularly shaped or overlapping. K-means performs best on spherical data.

**Regional Differences:** Countries with diverse cultural and social structures may report similar happiness scores despite having very different GDP levels, leading to overlapping

clusters (e.g., Scandinavian countries with high happiness and African nations with modest GDP but high social support).

```
# Calculate the silhouette score
silhouette_avg = silhouette_score(X, data_2019['Cluster'])
print(f'Silhouette Score: {silhouette_avg}')
```

 Silhouette Score: 0.4472718292353524

## 2. DBSCAN

The best silhouette score obtained was 0.6842 after changing the values of min-samples and the eps ranging from 0.2 to 0.35 and 5-8 respectively. Since Silhouette Score ranges from ranges from -1 (incorrect clustering) to +1 (highly dense clustering), with scores around zero indicating overlapping clusters, a score above 0.5 indicates a good clustering quality, suggesting that the clusters are reasonably well-separated and distinct.

```
# Calculating the silhouette score
score = silhouette_score(X_filtered, labels_filtered)
print(f'Silhouette Score for DBSCAN: {score}')
```

Silhouette Score for DBSCAN: 0.6842473673132903

Possible reasons:

- Optimal Parameter Selection (eps and min\_samples)  
The choice of parameters eps and min\_samples is crucial in DBSCAN as these parameters can directly influence the clustering results and the algorithm's ability to identify meaningful clusters. With the right values, DBSCAN can effectively detect dense regions while ignoring noise, resulting in compact clusters and high silhouette scores.
- Distinct Clusters in Happiness and GDP Data  
The dataset likely contains distinct countries with similar levels of happiness and GDP per capita. For example, wealthier countries with higher happiness scores may naturally form a separate cluster from lower-income countries with lower scores which could lead to high silhouette scores since clusters are well separated.
- Noise filtering  
DBSCAN labels some points as noise (-1) which are excluded from silhouette calculation. By removing these outlier points, silhouette score only measures the compactness and separation of well defines clusters.

## 3. Bayes Net

The silhouette score of 0.4093 suggests that the clustering is reasonable, but there's room for improvement, indicating moderate clustering performance.

- Probabilistic dependencies  
The method uses probabilistic dependencies between variables, which can result in clusters that may be less distinct compared to distance-based methods, reducing their distinctiveness.
- Subtle similarities  
The method may group data points with more subtle similarities, resulting in overlapping clusters.

```
silhouette_avg = silhouette_score(X_scaled, df['cluster'])  
print(f'Silhouette Score: {silhouette_avg}')
```

Silhouette Score: 0.40931489727247883

## Davies-Bouldin Index (DBI) Comparison:

DBI is used to evaluate the quality of clustering algorithms. Since DBI Score ranges from ranges from 0 (perfect clustering) to  $+\infty$  (highly overlapped clustering), a lower score indicates better clustering quality. A score below 0.5 suggests good clustering quality, indicating that the clusters are reasonably well-separated and distinct.

### 1. K-means

We obtained a value of 0.74 for DBI, which is considered to be a moderate score.

Possible Reasons:

Presence of outliers: Outliers can often skew the clustering of the data, which might result in a moderate DBI score for K-means.

Variability in clusters: If cluster sizes differ (i.e. some are tightly packed, and some are more dispersed), the DBI score could increase.

Separation: If the clusters are not well-separated, the DBI score could also increase.

```
# Calculate the Davies-Bouldin Index  
from sklearn.metrics import davies_bouldin_score  
dbi = davies_bouldin_score(X, data_2019['Cluster'])  
print(f'Davies-Bouldin Index for 3 clusters: {dbi}')
```

Davies-Bouldin Index for 3 clusters: 0.7461020768092844

## 2. DBSCAN

The best Davies-Bouldin Index for DBSCAN obtained was 0.3804.

```
# Calculating the Davies-Bouldin Index
dbi = davies_bouldin_score(X_filtered, labels_filtered)
print(f'Davies-Bouldin Index for DBSCAN: {dbi}')
```

```
Davies-Bouldin Index for DBSCAN: 0.38048973681639453
```

Possible reasons:

- Compactness of clusters  
In the dataset, countries with similar Happiness Scores and GDP values are grouped together, resulting in tightly packed clusters.
- Exclusion of Noise Points  
DBSCAN assigns some data points as noise, effectively filtering out outliers. By focusing on well-defined clusters and excluding noisy data which leads to a lower DBI score.

## 3. Bayes Net

The DBI score of 0.8415 indicates that the clusters are moderately separated. The overall quality of clustering is strong with minimal overlap between clusters. There is room for further improvement (closer to 0) as a DBI score  $< 0.5$  indicates very good clustering, but the clusters are reasonably distinct.

Possible reasons:

- Less compact or more similar clusters  
The model may capture complex relationships in the data but fail to create well-defined clusters, leading to greater overlap between them. This means that the clusters may be less compact, meaning data points within clusters are more spread out or that the clusters may be more similar to each other, indicating less separation between clusters.
- Probabilistic nature of Bayesian Networks  
The probabilistic aspect of Bayesian Networks may introduce variability in cluster assignments, making clusters less distinct.

```
dbi = davies_bouldin_score(X_filtered, labels_filtered)
print(f'Davies-Bouldin Index: {dbi}')
```

Davies-Bouldin Index: 0.8415636422906996

## Recommendations:

Following is the performance of 3 clustering methods on Dataset 2.

| Method    | Silhouette Score    | Davies-Bouldin Index |
|-----------|---------------------|----------------------|
| K-means   | 0.4472718292353524  | 0.7461020768092844   |
| DBSCAN    | 0.6842473673132903  | 0.38048973681639453  |
| Bayes Net | 0.40931489727247883 | 0.8415636422906996   |

Based on the above scores of Dataset 2, DBSCAN has the highest silhouette and DBI score while Bayes Net has the lowest silhouette and DBI score. Since only DBSCAN has given a reasonable silhouette score of 0.5 and above and DBI score of 0.5 and below, it implies that DBSCAN is the most suitable clustering method among all.

### 1) Alternative Clustering Algorithms:

Other methods such as Gaussian Mixture Models, Agglomerative Hierarchical or Spectral Clustering can be considered for better clustering as they may be more suitable depending on the dataset characteristics.

### 2) Using other features of the dataset:

We chose 'GDP per capita' as the feature to do the clustering based on. However, we recognize that other features may also yield distinct and compact clusters.

For example, we tried to use 'Perceptions of corruption' instead of GDP to do Bayesian Network Clustering. Both the Silhouette Score and DBI improved.

```
silhouette_avg = silhouette_score(X_scaled, df['cluster'])
print(f'Silhouette Score: {silhouette_avg}')
```

Silhouette Score: 0.5430026544938341

The improvement of Silhouette Score from 0.41 to 0.54 shows that changing features improved the clustering results significantly.

```
# Calculating the Davies-Bouldin Index
dbi = davies_bouldin_score(X_filtered, labels_filtered)
print(f'Davies-Bouldin Index: {dbi}')
```

Davies-Bouldin Index: 0.6445230714980615

The improvement of DBI from 0.84 to 0.64 also shows that changing features improved the clustering results significantly.

Similarly, on using 'Healthy life expectancy' instead of GDP, both the Silhouette Score and DBI improved.

```
silhouette_avg = silhouette_score(X_scaled, df['cluster'])
print(f'Silhouette Score: {silhouette_avg}')
```

Silhouette Score: 0.5336783623437046

The improvement of Silhouette Score from 0.41 to 0.53 shows that changing features improved the clustering results significantly.

```
# Calculating the Davies-Bouldin Index
dbi = davies_bouldin_score(X_filtered, labels_filtered)
print(f'Davies-Bouldin Index: {dbi}')
```

Davies-Bouldin Index: 0.6311564724572126

The improvement of DBI from 0.84 to 0.63 also shows that changing features improved the clustering results significantly.

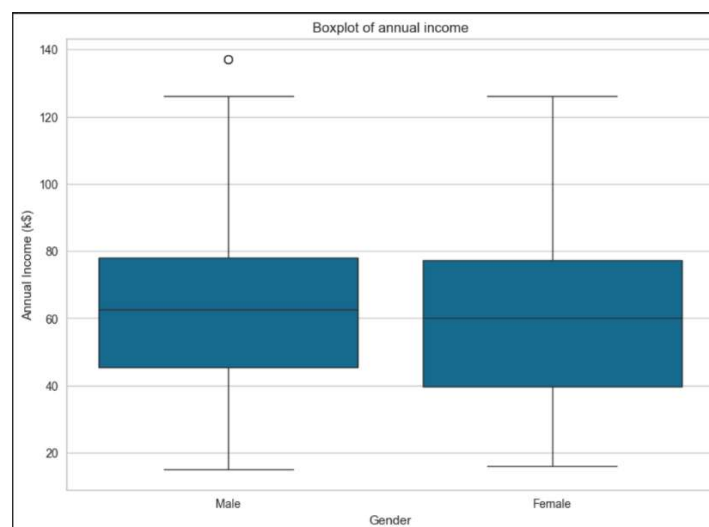
### 3) Adjust outliers:

Outliers might decrease the quality of clustering, so they can be adjusted. DBSCAN utilizes this method to filter out noise. This may have been a factor as to why this method showed more promising clustering results. Applying this on K-means as well as Bayes Networks could yield a higher Silhouette Score and a lower DBI.



## DATASET 3

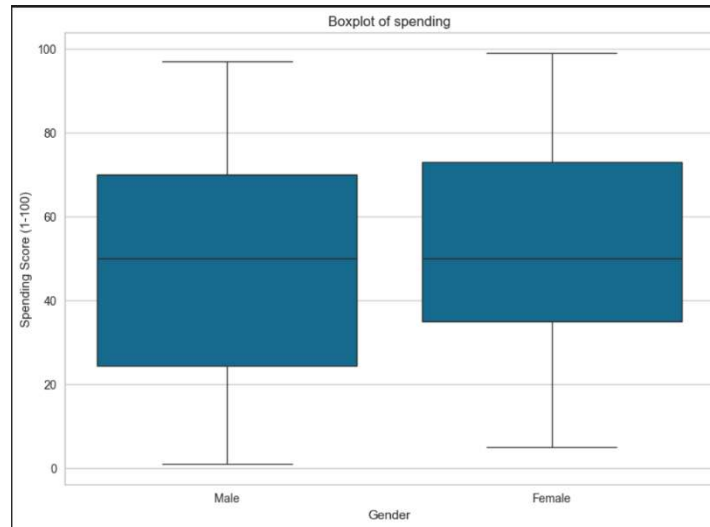
### Exploratory Data Analysis



The plot above shows the boxplot of the distribution of annual income between genders. We notice that their distributions are very similar therefore we can conclude that the genders do not have significant impact on the annual income. The mean and median values are also shown here.

```
Mean male income: 62.22727272727273
Median male income: 62.5
Mean female income: 59.25
Median female income: 60.0
```

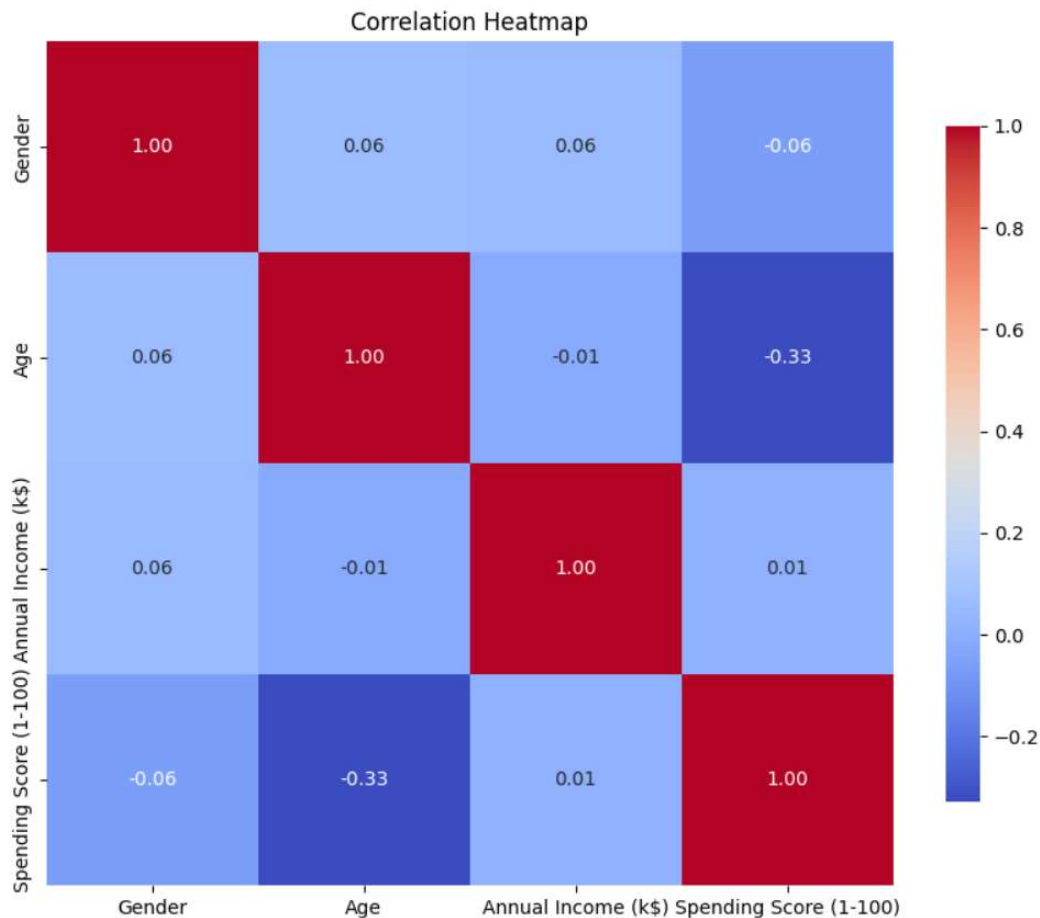




The plot above shows the boxplot of the distribution of spending power between genders. From here we also notice that once again there are no significant differences in spending power between genders. The mean and median values are also shown here.

```
Mean male spending: 62.22727272727273
Median male spending: 62.5
Mean female spending: 59.25
Median female spending: 60.0
```

Additionally, we also plotted a heatmap of the correlations between each variable. For this step we dropped CustomerID because it was not essential to our analysis and encoded the Gender column with LabelEncoder.



The correlation heatmap indicated that there are no significant correlations between any 2 variables. Lastly, now we can exclude 'Gender' from our analysis since it is non numerical, it also contributes relatively little impact towards spending power and annual income.

## Approach 1: KMeans Clustering

### 1. Scaling the data

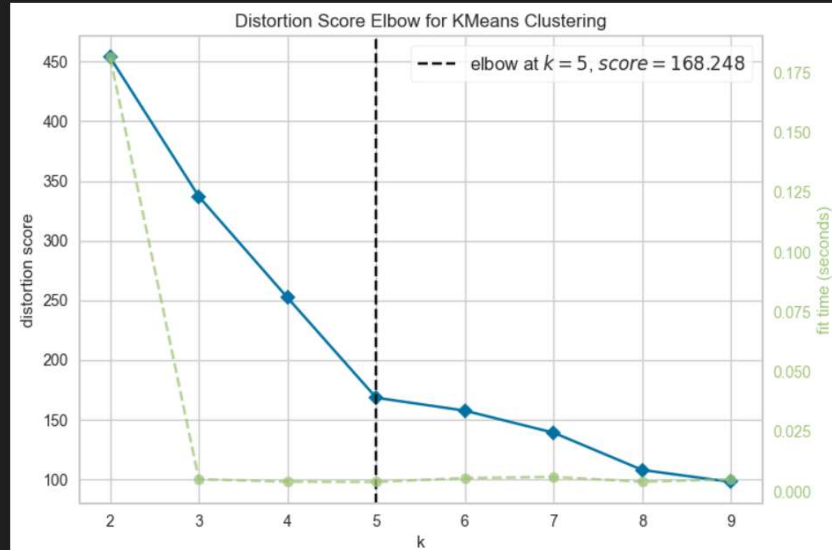
We first scale the numeric columns Age, Spending Power and Annual Income using StandardScaler() to reduce the impacts of their respective magnitudes and ranges of values which can cause unequal contribution between variables during our clustering.

### 2. Determining the no. of clusters k

To determine the right number of clusters to maximize performance of our KMeans model, we plot an inertia vs clusters graph to observe the elbow point of the graph where the rate of decrease of inertia drops rapidly.

```
# We use elbow method to choose number of clusters
model = KMeans(random_state=1)
visualizer = KElbowVisualizer(model, k=(2,10))

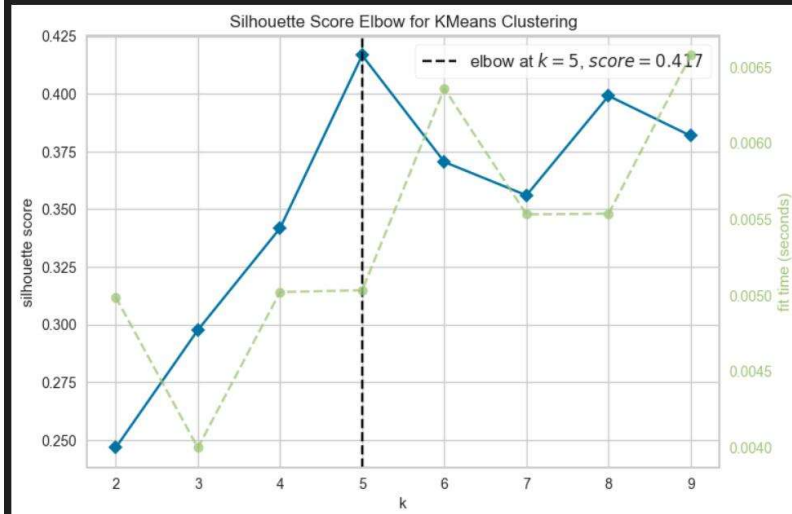
visualizer.fit(numeric_df_scaled)
visualizer.show()
plt.show()
✓ 0.7s
```



From the graph, we can clearly see the elbow point of the graph at k=5. To further confirm we can also observe the peak of silhouette score for each no. of clusters to determine the optimal no. of clusters.

```
model = KMeans(random_state=1)
visualizer = KElbowVisualizer(model, k=(2,10), metric='silhouette')

visualizer.fit(numeric_df_scaled)
visualizer.show()
plt.show()
✓ 0.2s
```



Using this method also yields  $k=5$  as the optimal no. of clusters hence we continue with  $k=5$ .

### 3. Initializing and fitting the model

To minimize time taken and maximize efficiency, we initialized the model using KMeans++ which works by starting the centroids from furthest points from each other first then working closer to the optimal positions. This method reduces the risk of converging to poor local minima which can happen in K-Means, since centroid initialization is random hence results may be unreliable at each iteration.

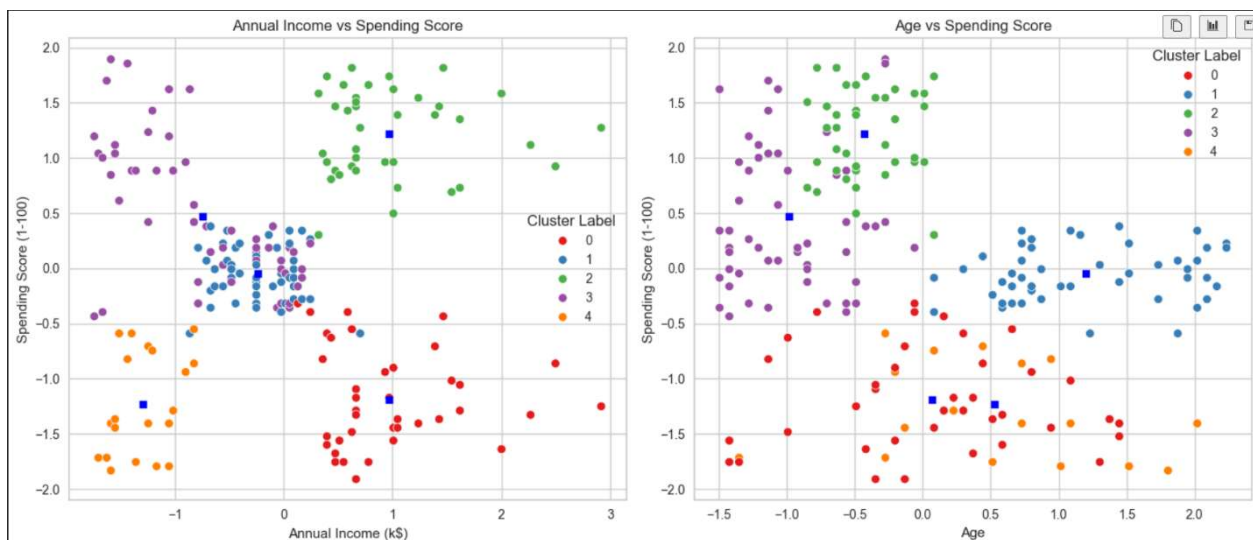
After which we can fit the data into the model to obtain cluster labels of each data point and their centroids to be visualized later.

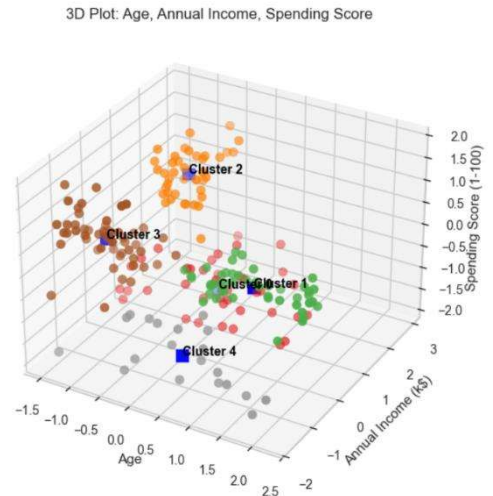
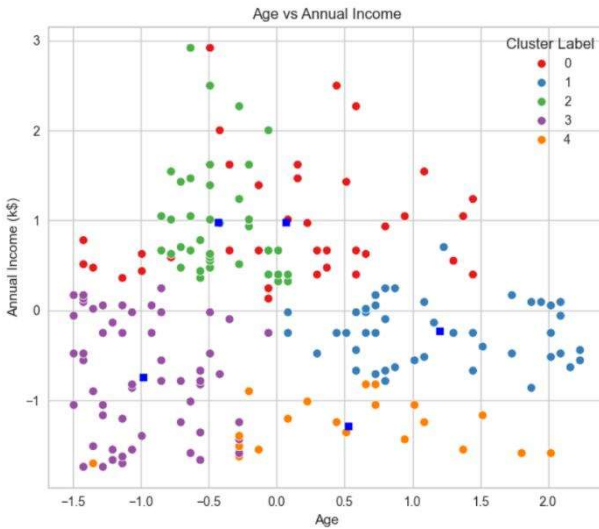
```
# Use kmeans++ initiation to help speed up clustering
kmeans_model = KMeans(n_clusters=5, init='k-means++').fit(numeric_df_scaled)

# Obtain cluster labels and center
cluster_labels = kmeans_model.labels_
centers = kmeans_model.cluster_centers_
```

### 4. Visualizing Clusters

We can now plot scatter plots to visualize the overall cluster. Since  $k=5$  we have 5 clusters representing different categories of users.





We also plotted three 2D graphs to allow us to see the clusters from different dimensions to gain insights to the cluster behaviors for each relationship.

## 5. Cluster Analysis

| Cluster Size  |    |
|---------------|----|
| Cluster Label |    |
| 0             | 39 |
| 1             | 47 |
| 2             | 40 |
| 3             | 54 |
| 4             | 20 |

The sizes of each cluster are calculated, and we can observe that the biggest cluster belongs to the group with relatively low annual income with medium spending and younger age, while the smallest group belongs to the group with low annual income with low spending and medium age.

To evaluate the clusters, we can calculate the Silhouette Score and Davies-Bouldin Score.

```
from sklearn.metrics import silhouette_score

# Separate features and cluster labels
features = numeric_df_scaled_clustered[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
labels = numeric_df_scaled_clustered['Cluster Label']

# Calculate the silhouette score
score = silhouette_score(features, labels)

print("Overall Silhouette Score:", score)
```

✓ 0.0s

Overall Silhouette Score: 0.41664341513732767

```
# Calculate DBI score
dbi_score = davies_bouldin_index(numeric_df, cluster_labels)
print(f'DBI Score: {dbi_score}')
```

✓ 0.0s

DBI Score: 0.8268598268441363

The scores obtained are moderate, though there are some possible reasons limiting its performance.

## 6. Limitations

- a. There are many overlapping points from different clusters, this is most clearly seen on the plot of Spending Score vs Annual Income where numerous points from cluster 1 and 3 overlap causing fuzzy boundaries between clusters.
- b. KMeans clustering works best with spherical clusters however, as seen from the plots cluster 4 is very sparsely distributed, hence KMeans may not be suitable for this.

## 7. Suggestions

We can explore other clustering methods which may allow us to cluster with other shapes that better match the dataset like Agglomerative clustering. Outliers in the dataset may influence the clustering negatively especially when KMeans is highly sensitive to outliers hence we should remove outliers.

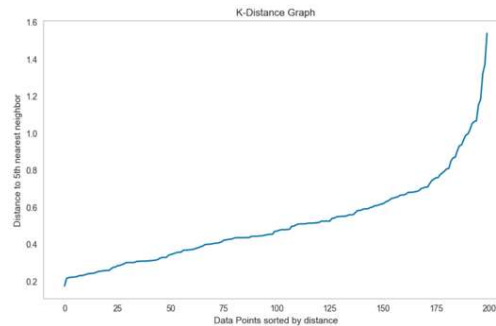
## Approach 2 : DBSCAN

Since we use the same scaled dataset, we can go straight to finding best parameters.

### 1. Obtaining optimal parameters

To obtain the optimal eps value, we conducted 2 methods. Using k-distance graph and using best silhouette score.

### 1.1. Using K-Distance Graph



In this case we used  $k=5$  to find distance to 5<sup>th</sup> nearest neighbor for each point and from what we can see, the knee point of the graph lies around the 170<sup>th</sup> data point which corresponds to around 0.7 hence we should choose a slightly lower eps value as standard practice like 0.5-0.6.

### 1.2. Using Silhouette Score

```
# potential values of eps and min_samples
eps = np.arange(0.1,5,0.1)
min_samples = np.arange(1,6)
```

```
# Sort by the silhouette score to find the best combination of parameters
df = pd.DataFrame.from_records(param_combination, columns=['Eps', 'Min_samples'])
df['Silhouette Score'] = silhouette
df['No of Clusters'] = no_of_clusters

df_sorted= df.sort_values(by='Silhouette Score', ascending=False)

# Filter out combinations that only gives 2 clusters (including outlier cluster)
df_sorted[df_sorted['No of Clusters']>2]
```

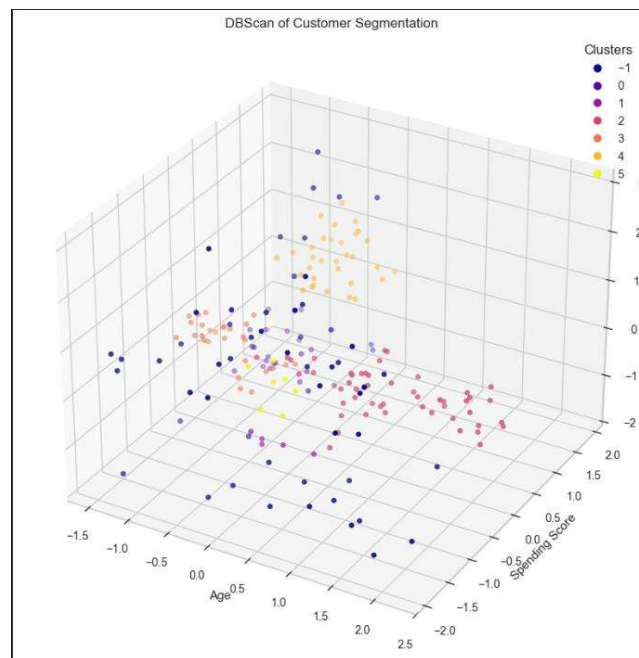
✓ 0.0s

|    | Eps | Min_samples | Silhouette Score | No of Clusters |
|----|-----|-------------|------------------|----------------|
| 29 | 0.6 | 5           | 0.188038         | 3              |
| 24 | 0.5 | 5           | 0.184514         | 7              |
| 27 | 0.6 | 3           | 0.177936         | 4              |



Using this method, we calculate the silhouette scores of every possible combination of eps and min\_sample in the range 0.1 to 5 and 1 to 6 respectively. However, most results may suggest having only 2 clusters, already including outliers thus effectively having only 1 optimal cluster. This is undesirable hence we filtered out only combinations that gives us at least 2 clusters not including outliers. Hence, landing on eps of 0.5 as previously estimated and min\_sample 5 since it results in sufficient number of clusters.

## 2. Cluster Analysis



The noise points are labeled as -1 and from here we can see that there are numerous points classified as noise points which may hinder the performance of DBSCAN. We then calculated silhouette score and DBI score excluding the noise points.

```
# Calculating the silhouette score
score = silhouette_score(X_filtered, labels_filtered)
print(f'Silhouette Score for DBSCAN: {score}')
```

✓ 0.0s

---

Silhouette Score for DBSCAN: 0.48171491786230414



```
# Calculating the Davies-Bouldin Index
dbi = davies_bouldin_score(x_filtered, labels_filtered)
print(f'Davies-Bouldin Index for DBSCAN: {dbi}')
✓ 0.0s
Davies-Bouldin Index for DBSCAN: 0.638574875248808
```

### 3. Limitations

- a. The silhouette score obtained is only moderate due to overlapping data between clusters causing poor intra-cluster compactness and weak inter-cluster separation.
- b. Having large number of noise points can also lead to poor silhouette scores even though it is calculated without noise points because it can lead to less distinct and sparse cluster formations hence DBScan cannot sufficiently separate the data into meaningful clusters.
- c. At the same time, this may also be due to presence of outliers which alters the calculation of eps and min\_samples in the first place.

### 4. Suggestions

Remove outliers to ensure they do not affect the calculation of eps and min\_samples parameters. It may also help to reduce the number of points classified as noise points, allowing more distinct clusters to be formed.

### Inference

Comparing KMeans and DBScan, DBScan has both a higher silhouette score and a lower Davies-Bouldin score, though the scores are still not ideal, it is still performing better than KMeans. Hence showing that DBScan is more appropriate for this dataset

### Conclusion

The choice of clustering algorithm is crucial and should be guided by the specific characteristics of the dataset and the objectives of the analysis. Each clustering method presents unique strengths and weaknesses, making them suitable for different scenarios.

K-Means is a popular choice due to its speed and simplicity, particularly effective for datasets with spherical clusters. Its efficiency, however, is counterbalanced by its assumptions regarding cluster shape and its sensitivity to outliers. In cases where data is

more complex or contains significant noise, K-Means may yield suboptimal results, necessitating the exploration of alternative clustering techniques.

DBSCAN offers notable advantages in its ability to identify clusters of arbitrary shapes and manage noise, making it a versatile option for real-world data that often deviates from ideal conditions. Nevertheless, its performance can be significantly affected by the selection of parameters, and it may struggle with clusters of varying densities. This sensitivity can complicate its application, particularly in high-dimensional datasets where it may obscure meaningful cluster structures.

Bayesian Networks Clustering excels in capturing probabilistic relationships among variables and is particularly effective in scenarios where data may be incomplete. However, its computational intensity can limit its practicality, especially with smaller datasets or in applications requiring rapid analysis. The reliance on larger datasets for accurate modeling further emphasizes the need for careful consideration of data availability when employing this method.

In conclusion, the effectiveness of clustering methods is contingent upon the nature of the data and the specific requirements of the clustering task. A thorough understanding of each algorithm's capabilities and limitations is essential for selecting the most appropriate method. Future work should focus on developing hybrid approaches or adaptive algorithms that can leverage the strengths of multiple clustering techniques, thereby enhancing performance across diverse data scenarios.

## References

1. Dataset 1: NBA Shot. Mexwell. <https://www.kaggle.com/datasets/mexwell/nba-shots>
2. Dataset 2: United Nations Sustainable Development Solutions Network. (2019). World Happiness Report 2019. Retrieved from <https://www.kaggle.com/datasets/unsdsn/world-happiness?select=2019.csv>
3. Dataset 3: Mall Customer Segmentation Data. Vijay Choudhary. <https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python>

